

Lecture Note: IM Session 2 – Input, Type Casting & Arithmetic Operators

Nishut Suman

3 Jan, 2026 At 6:00 PM

Notes

Foundation

Recommended

Resource



Associated Lectures (1)



Description



Discussions



Lecture Notes: Input, Type Casting and Arithmetic Operators



Learning Objectives

By the end of this session, you will be able to:

- Use `input()` correctly
- Understand why input is always treated as text
- Convert data types using type casting
- Perform arithmetic operations safely
- Understand why modulo and exponentiation exist and when to use them



Big Idea: How Data Flows in a Program

A program is a flow of data:

Input comes from the user

Python processes the data

Output is shown to the user

If the data type is wrong at any step, the program breaks or behaves incorrectly.



1 Input and Output Flow

How `input()` Works

The `input()` function always returns a **string**, no matter what the user types.

Example:

```
age = input("Enter age: ")
print(age)
print(type(age))
```

Even if the user types `25`, Python stores it as:

```
"25"
```

Why Input Is Always a String

Computers cannot guess user intent.

If someone types:

```
123
```

Python cannot assume:

- Is it an age
- Is it a pin code
- Is it a year
- Is it text

So Python plays safe and treats everything as text.

The Common Bug

```
age = input("Enter age: ")
print(age + 1)
```

This breaks because:

- `age` is a string
- `1` is an integer
- Python does not mix text and numbers automatically

This is a **type mismatch error**.

2 Type Casting

Type casting means **explicitly telling Python what type you want**.

Python will not guess for you.

Common Casting Functions

Function	Converts to
<code>int()</code>	Integer
<code>float()</code>	Decimal number
<code>str()</code>	String

Correct Way to Handle Numeric Input

```
age = int(input("Enter age: "))  
print(age + 1)
```

Now Python knows:

- Convert user input to integer
- Then perform arithmetic

Why Python Forces Explicit Casting

Python prioritizes:

- Safety
- Clarity
- Predictable behavior

Silent conversion can cause hidden bugs. Explicit casting makes your intention clear.

Another Example

```
price = float(input("Enter price: "))  
quantity = int(input("Enter quantity: "))  
total = price * quantity  
print(total)
```

Each value is converted deliberately.

3 Arithmetic Operators

Arithmetic operators allow Python to perform calculations.

Basic Operators

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division

Example:

```
a = 10
b = 3

print(a + b)
print(a - b)
print(a * b)
print(a / b)
```

Floor Division //

Floor division removes the decimal part.

```
print(10 // 3)
```

Output:

```
3
```

Why this exists:

- Useful when dividing items into equal groups
- Used heavily in indexing, pagination, and batching

Example:

- 23 students divided into groups of 5
- `23 // 5` gives number of full groups

Modulo %

Modulo gives the **remainder** after division.

```
print(10 % 3)
```

Output:

```
1
```

Why Modulo Exists

Modulo answers questions like:

- Is a number even or odd
- How many items are left
- Cycle based logic

Even or Odd Check

```
num = int(input("Enter number: "))  
print(num % 2)
```

- Remainder `0` means even
- Remainder `1` means odd

This is a core logic tool in programming.

Exponentiation **

Exponentiation raises a number to a power.

```
print(2 ** 3)
```

Output:

```
8
```

Meaning:

- 2 multiplied by itself 3 times

Why Exponentiation Exists

Used for:

- Squares and cubes
- Mathematical formulas
- Growth calculations
- Scientific and AI computations

Examples:

```
print(5 ** 2)  # square
print(3 ** 3)  # cube
```

4 Practice Exercises

Exercise 1: Simple Calculator

Requirements

- Take two numbers from user
- Print:
 - Sum
 - Difference
 - Product
 - Division
 - Remainder
 - Power

Sample Solution

```
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

print("Sum:", num1 + num2)
print("Difference:", num1 - num2)
print("Product:", num1 * num2)
print("Division:", num1 / num2)
```

```
print("Remainder:", num1 % num2)
print("Power:", num1 ** num2)
```

Exercise 2: Even or Odd Checker

Task

- Input a number
- Print the remainder when divided by 2

Sample Solution

```
num = int(input("Enter a number: "))
print("Remainder:", num % 2)
```

Interpretation:

- Remainder 0 means even
- Remainder 1 means odd

Key Takeaways

Concept	Meaning
<code>input()</code>	Always returns string
Type casting	Explicit conversion
Arithmetic operators	Perform calculations
Floor division	Removes decimals
Modulo	Finds remainder
Exponentiation	Powers and growth

Understanding **why** these tools exist is more important than memorizing them.

Final Thought

Python does exactly what you tell it. It never guesses.

Once you control data types, you control program behavior.

